TASK 23:ASSIGNMENT NO 23

Name of Student : Anuja Vilas Wankhade

Domain: *Artificial intelligence and machine Learning (AIML)*

College Name: Manav School of Polytechnic AKOLA

Branch : Computer Engineering

GitHub Repository: https://github.com/anujaw193-star/python-project-collection

---

1. Linear Regression

Code & Output:

```
[2]
✓ 0s
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import r2_score
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score, classification_report
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
import joblib
```

```
[10]
✓ 0s
from sklearn import preprocessing
from pandas.core.arrays import categorical
# Q1: Linear Regression
# Load Dataset
df = pd.read_csv("insurance.csv")
# Display Dataset
print("First 5 Rows:")
print(df.head())

print("\nDataset Shape:")
print(df.shape)

print("\nDateset Information:")
print(df.info())

print("\nStatistical Summary:")
print(df.describe())

print("\nStatistical Summary:")
print(df.describe())

print("\nMissing Values:")
print(df.isnull().sum())
```

```
First 5 Rows:
   age     sex     bmi  children smoker     region      charges
0   19  female  27.900         0    yes  southwest  16884.92400
1   18    male  33.770         1     no  southeast   1725.55230
2   28    male  33.000         3     no  southeast   4449.46200
3   33    male  22.705         0     no  northwest  21984.47061
4   32    male  28.880         0     no  northwest   3866.85520

Dataset Shape:
(1338, 7)

Dateset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       1338 non-null   int64
 1   sex       1338 non-null   object
 2   bmi       1338 non-null   float64
 3   children  1338 non-null   int64
 4   smoker    1338 non-null   object
 5   region    1338 non-null   object
 6   charges   1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
None
```

```
Statistical Summary:
                age          bmi     children       charges
count  1338.000000  1338.000000  1338.000000   1338.000000
mean     39.207025    30.663397     1.094918  13270.422265
std      14.049960     6.098187     1.205493  12110.011237
min      18.000000    15.960000     0.000000   1121.873900
25%      27.000000    26.296250     0.000000   4740.287150
50%      39.000000    30.400000     1.000000   9382.033000
75%      51.000000    34.693750     2.000000  16639.912515
max      64.000000    53.130000     5.000000  63770.428010

Statistical Summary:
                age          bmi     children       charges
count  1338.000000  1338.000000  1338.000000   1338.000000
mean     39.207025    30.663397     1.094918  13270.422265
std      14.049960     6.098187     1.205493  12110.011237
min      18.000000    15.960000     0.000000   1121.873900
25%      27.000000    26.296250     0.000000   4740.287150
50%      39.000000    30.400000     1.000000   9382.033000
75%      51.000000    34.693750     2.000000  16639.912515
max      64.000000    53.130000     5.000000  63770.428010

Missing Values:
age         0
sex         0
bmi         0
children    0
smoker      0
region      0
charges     0
dtype: int64
```

```
[11] ✓ 0s
# Features and Target
X = df.drop("charges", axis=1)
y = df["charges"]

[12] ✓ 0s
# Categorical Columns
categorical_features = ["sex", "smoker", "region"]

[13] ✓ 0s
# Preprocessing
preprocessor = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(drop="first"), categorical_features)
    ],
    remainder="passthrough"
)

[14] ✓ 0s
# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

[15] ✓ 0s
# Create Model
model = Pipeline([
    ("preprocessor", preprocessor),
    ("regressor", LogisticRegression())
])
```

```
print(y_train.head())
print(y_train.dtype)
print(y_train.unique()[:10])
```

```
560      9193.83850
1285     8534.67180
1142    27117.99378
969      8596.82780
486     12475.35130
Name: charges, dtype: float64
float64
[ 9193.8385   8534.6718  27117.99378  8596.8278  12475.3513  13405.3903
  2150.469   13747.87235  6610.1097  39047.285  ]
```

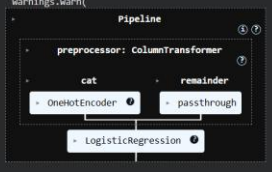
```
df["High_Charges"] = (df["charges"] > df["charges"].median()).astype(int)
X = df.drop(["charges", "High_Charges"], axis=1)
y = df["High_Charges"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Train Model
model.fit(X_train, y_train)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
/usr/local/lib/python3.12/dist-packages/sklearn/compose/_column_transformer.py:1667: FutureWarning:
The format of the columns of the 'remainder' transformer in ColumnTransformer.transformers_ will change in version 1.7 to match the format of the other transformers.
At the moment the remainder columns are stored as indices (of type int). With the same ColumnTransformer configuration, in the future they will be stored as column names (of type str).
To use the new behavior now and suppress this warning, use ColumnTransformer(force_int_remainder_cols=False).

  warnings.warn(
```




```
[23]
✓ 0s
# Prediction
y_pred = model.predict(X_test)
```

```
[24]
✓ 0s
# R2 Score
r2 = r2_score(y_test, y_pred)
print("R2 Score:", r2)

# Actual vs Predicted Values
result = pd.DataFrame({'Actual': y_test.values, 'Predicted': y_pred})
print("\nFirst 10 Predictions:")
print(result.head(10))
```

```
R2 Score: 0.6388951268807546

First 10 Predictions:
   Actual  Predicted
0       0          1
1       0          0
2       1          1
3       0          0
4       1          1
5       0          0
6       0          0
7       1          1
8       0          0
9       1          1
```

2. Logistic Regression
   Code & Output:

```
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.9104477611940298
Precision: 0.8828125
Recall: 0.9262295081967213
F1 Score: 0.904

Classification Report:
              precision    recall  f1-score   support

           0       0.94      0.90      0.92       146
           1       0.88      0.93      0.90       122

    accuracy                           0.91       268
   macro avg       0.91      0.91      0.91       268
weighted avg       0.91      0.91      0.91       268
```

3. K-Nearest Neighbors

Code:

```
# Try Different K Values

for k in [3,5,7]:
  model = Pipeline([
      ("preprocessor", preprocessor),
      ("classifier", KNeighborsClassifier(n_neighbors=k))
  ])
  model.fit(X_train, y_train)
  y_pred = model.predict(X_test)
  acc = accuracy_score(y_test, y_pred)
  accuracy_list.append((k, acc))
  print(f"\nK = {k}")
  print("Accuracy:", acc)

  # Best K
  best_k = max(accuracy_list, key=lambda x: x[1])
  print("\n==================")
  print("\nBest K:", best_k[0])
  print("Best Accuracy:", best_k[1])
```

Output:

```
...
    K = 3
    Accuracy: 0.8843283582089553

    ==================

    Best K: 3
    Best Accuracy: 0.8843283582089553

    K = 5
    Accuracy: 0.8880597014925373

    ==================

    Best K: 5
    Best Accuracy: 0.8880597014925373

    K = 7
    Accuracy: 0.8955223880597015

    ==================

    Best K: 7
    Best Accuracy: 0.8955223880597015
```

4. Naïve Bayes

Code:

```python
# Naive Bayes Model
model = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", GaussianNB())
])

# Train Model
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Evaluation
print("\nConfusion Matrix:")
print(confusion_matrix(y_test,y_pred))

print("\nAccuracy:", accuracy_score(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Output:

```
Confusion Matrix:
[[146   0]
 [ 68  54]]

Accuracy: 0.746268656716418

Classification Report:
              precision    recall  f1-score   support

           0       0.68      1.00      0.81       146
           1       1.00      0.44      0.61       122

    accuracy                           0.75       268
   macro avg       0.84      0.72      0.71       268
weighted avg       0.83      0.75      0.72       268
```

5. Algorithm Comparison

Code:

```python
# Models
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "KNN": KNeighborsClassifier(n_neighbors=7),
    "Naive Bayes": GaussianNB()
}

results = []

for name, classifier in models.items():
  model = Pipeline([
      ("preprocessor", preprocessor),
      ("classifier", classifier)
  ])
  model.fit(X_train, y_train)
  y_pred = model.predict(X_test)

  results.append({
      "Algorithm": name,
      "Accuracy": accuracy_score(y_test, y_pred),
      "Precision": precision_score(y_test, y_pred),
      "Recall": recall_score(y_test, y_pred),
      "F1 Score": f1_score(y_test, y_pred)
  })

comparison = pd.DataFrame(results)

print("\nAlgorithm Comparison:")
print(comparison)

best = comparison.loc[comparison["Accuracy"].idxmax()]
print("\nBest Algorithm:")
```

```python
print("\nAlgorithm Comparison:")
print(comparison)

best = comparison.loc[comparison["Accuracy"].idxmax()]
print("\nBest Algorithm:")
print(best)
```

Output:

```
...
    Algorithm Comparison:
                 Algorithm  Accuracy  Precision    Recall  F1 Score
    0  Logistic Regression  0.910448   0.882812  0.926230  0.904000
    1                  KNN  0.895522   0.891667  0.877049  0.884298
    2          Naive Bayes  0.746269   1.000000  0.442623  0.613636

    Best Algorithm:
    Algorithm    Logistic Regression
    Accuracy                0.910448
    Precision               0.882812
    Recall                   0.92623
    F1 Score                   0.904
    Name: 0, dtype: object
```